

Activity 2 — Web Scraping with Python

What you'll learn

Web scraping means pulling data straight off a web page with code. You'll scrape the **CAR Region Open Data Portal** — a live web page with three tables (weather stations, schools, and tourism sites) — turn them into clean tables, and build an analysis dashboard.

The page we scrape: https://datasciencebaguio-production.up.railway.app/car_data
Honest note: this portal holds **simulated sample data** made for this workshop. The numbers are realistic but invented — not official government figures. What's real is the **skill**.

Before you start

- A **Google account** to use Google Colab.
- An **internet connection** (we pull the page live — no file to upload).

Run each cell with **Shift + Enter**, top to bottom.

Step 1 — Install the scraping tools

COPY THIS CODE

```
!pip install beautifulsoup4 requests
```

You should see: install messages or "Requirement already satisfied".

Step 2 — Load the libraries

COPY THIS CODE

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

print("Libraries loaded.")
```

You should see: Libraries loaded.

Step 3 — Download the web page

`requests.get` fetches the raw page from the portal. A status of **200** means success.

COPY THIS CODE

```
url = 'https://datasciencebagueio-production.up.railway.app/car_data'

try:
    response = requests.get(url, timeout=15)
    print(f"Status code: {response.status_code}")
    if response.status_code == 200:
        print("Page downloaded successfully.")
    else:
        print("Got a response, but not 200. The site may be busy.")
except Exception as e:
    print(f"Could not reach the site: {e}")
    print("Check your internet connection and try again.")
```

You should see: Status code: 200 and "Page downloaded successfully."

Step 4 — Parse the HTML and look at the page

BeautifulSoup turns the raw HTML into something we can search. We read the page heading, the notice, and count how many tables are on the page.

COPY THIS CODE

```

soup = BeautifulSoup(response.text, 'html.parser')

print("Page heading:", soup.find('h1').text)

notice = soup.find('p')
if notice:
    print("\nNotice on the page:")
    print(" ", notice.text.strip())

tables_on_page = soup.find_all('table')
print(f"\nThis page has {len(tables_on_page)} tables.")

print("\nSection titles found:")
for h2 in soup.find_all('h2'):
    print(" -", h2.text)

```

You should see: the page heading, the "simulated data" notice, "This page has 3 tables", and the three section titles.

Step 5 — Scrape all three tables at once

`pd.read_html` reads every table on the page into a list. This page has three.

COPY THIS CODE

```

tables = pd.read_html(response.text)
print(f"Scraped {len(tables)} tables from the page.\n")

weather_df = tables[0]
schools_df = tables[1]
tourism_df = tables[2]

print(f"Weather stations: {len(weather_df)} rows")
print(f"Schools: {len(schools_df)} rows")
print(f"Tourism sites: {len(tourism_df)} rows")

```

You should see: Weather **46**, Schools **50**, Tourism **40** rows.

Step 6 — Look at what we scraped

Always eyeball your data before analyzing it.

COPY THIS CODE

```
print("WEATHER STATIONS (first 5):")
display(weather_df.head())
print("SCHOOLS (first 5):")
display(schools_df.head())
print("TOURISM SITES (first 5):")
display(tourism_df.head())
```

You should see: the first 5 rows of each table.

Step 7 — Quick summary numbers

COPY THIS CODE

```
print("WEATHER")
print(f"  Average temperature: {weather_df['Temperature_C'].mean():.1f} C")
print(f"  Average rainfall: {weather_df['Rainfall_mm'].mean():.1f} mm")

print("\nSCHOOLS")
print(f"  Total enrollment: {schools_df['Enrollment'].sum():,} students")
print(f"  Average class ratio: {schools_df['Student_Teacher_Ratio'].mean():.1f} students per teacher")

print("\nTOURISM")
print(f"  Total annual visitors: {tourism_df['Annual_Visitors'].sum():,}")
print(f"  Average rating: {tourism_df['Rating'].mean():.2f} out of 5")
```

You should see: headline figures for each table (e.g. avg temperature ~24 C).

Step 8 — The analysis dashboard

Six charts in one figure, pulling from all three scraped tables.

COPY THIS CODE

```

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('CAR Region Data Dashboard (Simulated Sample Data)', fontsize=15,
fontweight='bold')

# 1. Temperature by province
weather_df.boxplot(column='Temperature_C', by='Province', ax=axes[0, 0])
axes[0, 0].set_title('Temperature by Province')
axes[0, 0].set_xlabel('')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Students by school type
by_type = schools_df.groupby('Type')['Enrollment'].sum()
axes[0, 1].pie(by_type.values, labels=by_type.index, autopct='%1.0f%%', startangle=90)
axes[0, 1].set_title('Students by School Type')

# 3. Rainfall vs elevation
axes[0, 2].scatter(weather_df['Elevation_m'], weather_df['Rainfall_mm'], alpha=0.6,
color='teal')
axes[0, 2].set_title('Rainfall vs Elevation')
axes[0, 2].set_xlabel('Elevation (m)')
axes[0, 2].set_ylabel('Rainfall (mm)')

# 4. Tourism ratings
axes[1, 0].hist(tourism_df['Rating'], bins=12, color='gold', edgecolor='black')
axes[1, 0].axvline(tourism_df['Rating'].mean(), color='red', linestyle='--',
label=f"Mean {tourism_df['Rating'].mean():.2f}")
axes[1, 0].set_title('Tourism Site Ratings')
axes[1, 0].set_xlabel('Rating')
axes[1, 0].legend()

# 5. Visitors by tourism type
by_t = tourism_df.groupby('Type')['Annual_Visitors'].sum().sort_values()
axes[1, 1].barh(by_t.index, by_t.values, color='coral')
axes[1, 1].set_title('Annual Visitors by Type')
axes[1, 1].set_xlabel('Visitors')

# 6. Weather conditions
cond = weather_df['Condition'].value_counts()
axes[1, 2].bar(range(len(cond)), cond.values, color='lightblue')
axes[1, 2].set_xticks(range(len(cond)))
axes[1, 2].set_xticklabels(cond.index, rotation=45, ha='right')
axes[1, 2].set_title('Weather Conditions')

```

```
axes[1, 2].set_ylabel('Stations')

plt.tight_layout()
plt.show()
print("Dashboard built from the scraped CAR data.")
```

You should see: a 6-chart dashboard (boxplot, pie, scatter, histogram, bar, bar).

Tips & common errors

- Run cells **top to bottom** with Shift + Enter.
- **Status not 200, or a connection error?** Check your internet and re-run Step 3. The portal may be waking up — try once more.
- **KeyError on a column?** Column names are spelling- and case-sensitive (e.g. Temperature_C, Annual_Visitors). Use them exactly.
- Always check a site's **terms** before scraping it, and always be clear whether data is **real or simulated**.

What's next

In **Activity 3** you'll take this scraped tourism data and run a full **exploratory data analysis**.